

SupaBase How-to

Bennet Fuhrmann

3596922

bennet.fuhrmann@fh-aachen.de

Stand: 28. Oktober 2025

VORWORT

SupaBase wird für unsere HSG-Lernwelt als Datenbank dienen und ebenfalls die Datenverwaltung und Account Steuerung übernehmen.

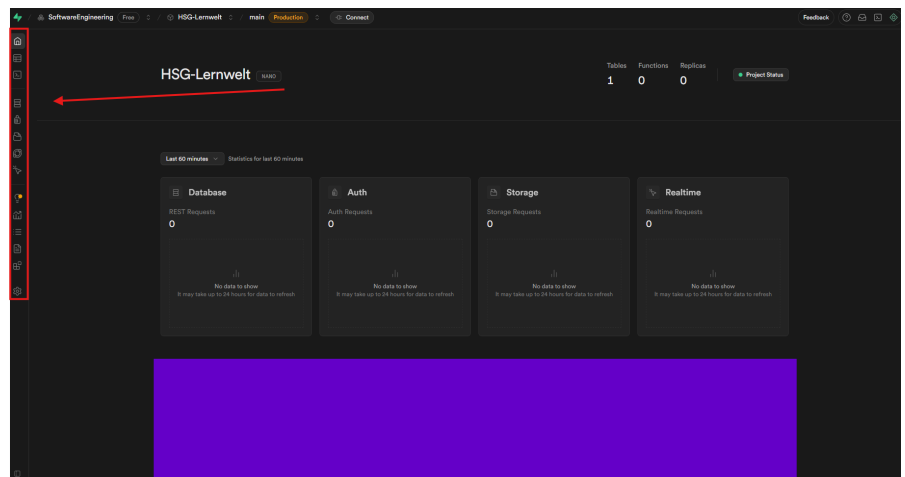
Damit wir alle Zugriff auf das bereits erstellte Projekt "HSG-Lernwelt" haben, werden wir im nächsten Weekly-Meeting die Accounts erstellen, bzw. die Einladung per Mail verschicken.

ERSTER ÜBERBLICK

In der Abbildung seht ihr die Startseite.

Der wichtigste Abschnitt (rot), ist unsere Navigationsleiste links. Der Teil, der in der Abbildung mit einem lila Balken verdeckt ist, ist vorerst unwichtig.

Die Startseite gibt uns auf einen Blick eine grobe Übersicht über unser Projekt und die Prozesse.



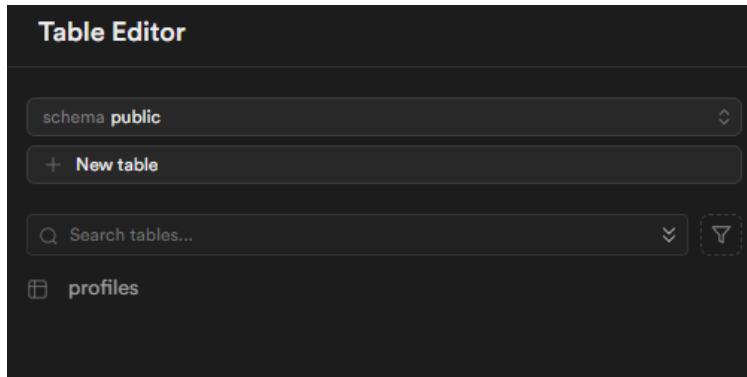
Im Folgenden wird diese Dokumentation Schritt-für-Schritt alle Abschnitte der Navigationsleiste durchgehen und beschreiben.

1 - Table Editor

Der Table Editor ist eine der Hauptfunktionen unserer Datenbank.

Man kann ihn sich wie eine Art Sammlung von Excel-Tabellen. Die Nutzung ist relativ intuitiv und ist vergleichbar mit einer Mischung aus Excel und Klassen in Programmiersprachen.

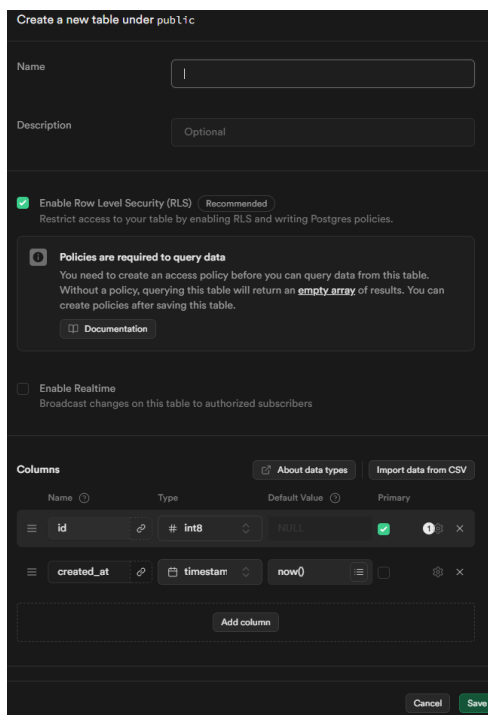
1.1 - Table Editor: Startseite



Auf der Startseite können wir als Erstes die Anzeige-Option wählen. Hier nutzen wir im Normalfall “public”. Dabei handelt es sich um Namespaces. Direkt darunter haben wir die Option einen neuen Table zu erstellen.

Unterhalb dieser Optionen, sehen wir alle bereits existierenden Tables und können sie dort im Detail ansehen und bearbeiten.

1.2 - Table Editor: Table erstellen



Um einen neuen Table zu erstellen, klicken wir auf “+New table” und sehen anschließend das Fenster in der

Name: Wir geben dem Table einen sinnvollen Namen.

Description: Optional eine kurze Beschreibung, damit den Sinn nachvollziehen können, falls dieser sich nicht aus dem Namen ergibt.

Enable RLS & Enable Realtime: Diese Haken bleiben immer wie auf der Abbildung.

Columns: Hier kommt die Analogie zu den Klassen ins Spiel: Das Vorgehen ist wie bei Attributen in C++/Java.

Name -> Name unserer Spalte

Typ -> Typ der zu speichernden Werte. (zB. bool, text)

Default Value -> Falls keine Neubelegung soll folgender Wert gespeichert sein.

Primary -> Wird meistens nur für die vorgegebene Column “id” genutzt und dient zur eindeutigen Identifizierung der Reihen (z.B. bei gleichen Namen von SuS). Er wird außerdem als “Foreign Key” in Relationen genutzt (später mehr dazu).

Zahnrad -> Vorerst unwichtig bzw. durch die Namen und Kurzbeschreibungen selbst-erklärend.

Wichtig: Das Attribut “id” bleibt in jedem erstellten Table zur Identifikation!

2 - Der SQL Editor

Der SQL Editor ist im Vergleich zum Table Editor das Terminal unserer Datenbank.

2.1 - SQL Editor: Wozu benutzen wir den Editor?

Der Table Editor ist gut, um schnell einzelne Zeilen zu checken oder einen Wert zu ändern. Sobald es komplex wird, müssen wir den SQL Editor nutzen.

Beispiele:

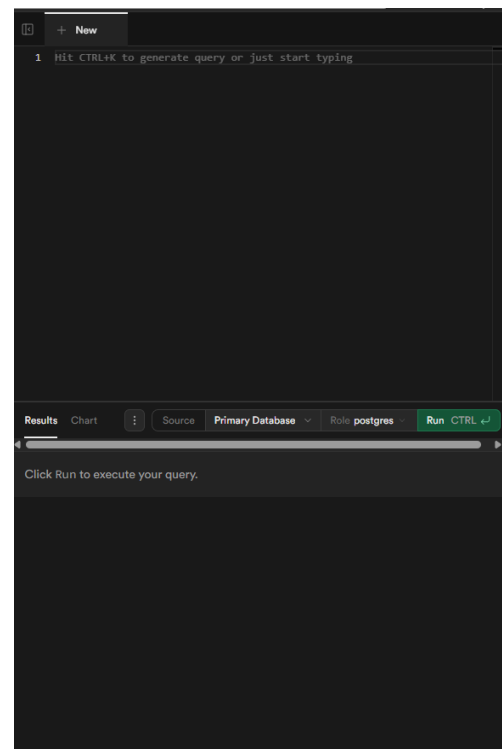
- *"Zeige mir alle Benutzer, die 'Max' heißen UND sich in den letzten 30 Tagen angemeldet haben."*
- *"Zähle, wie viele Antworten jeder einzelne User richtig hat."*
- *"Ändere bei allen 100 SuS den Status der Aufgabe 1 von 'active' auf 'false'."*

Solche Operationen, die mehrere Tabellen verknüpfen, genannt "*JOINS*", oder Daten filtern und zusammenfassen, genannt "*Aggregation*", gehen nur hier.

2.2 - SQL Editor: Der Aufbau

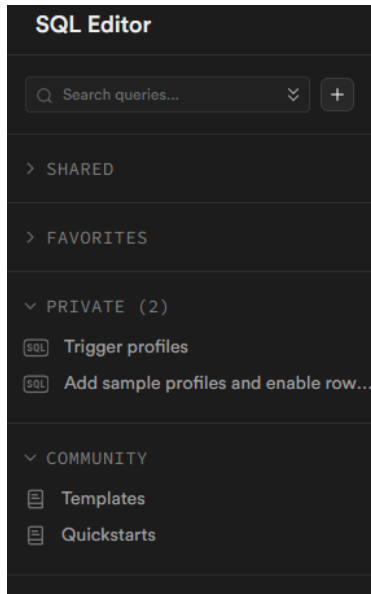
Das Fenster ist super simpel gehalten:

1. **Editor-Fenster (Mitte):**
Hier schreiben wir Queries hinein.
Ein klassischer Start ist:
`SELECT * FROM public.users;`
(Zeige mir [*] alles [FROM] aus der Tabelle [users]).
2. **"Run"-Button (Grün):**
Führt den Befehl aus, den man geschrieben hat.
3. **Ergebnis-Panel (Unten):**
Wenn man Daten abfragt (mit `SELECT`), erscheinen die Ergebnisse hier unten.
Wenn man was ändert (z.B. `UPDATE`), steht hier meist nur "Success".



2.3 - SQL Editor: Queries & Templates (Die Sidebar)

Das Nützlichste beim SQL Editor in Supabase ist die linke Sidebar:



- **Templates (Vorlagen):**
Supabase weiß, dass viele Befehle kompliziert sind. Hier findet ihr Vorlagen für gängige Aufgaben, z.B. um RLS-Policies (Sicherheitsregeln) zu erstellen oder um Funktionen zu bauen. Mega nützlich, vor allem, weil RLS-Policies für uns immer nötig sind, da wir damit die Zugriffsrechte usw. auf Tabellen klären. Außerdem müssen wir dann nicht ganz so tief in SQL tauchen :)
- **Queries (Gespeicherte Abfragen):** Hier kann man Befehle speichern. Wenn wir eine komplexe Abfrage haben, die wir häufiger gebrauchen könnten, speichert ihr sie hier ab und müsst sie nur noch anklicken, statt sie neu zu tippen.

3 - Der Database-Bereich

Der Database-Bereich im Menü ist der Sammelpunkt für die Editoren, aber er enthält noch viel mehr. Hier verwalten wir die Datenbank *selbst*.

Analogie:

Wenn der Table Editor das Wohnzimmer ist, in dem wir Möbel (Daten) verrücken, ist der Database-Bereich der Keller mit dem Sicherungskasten und den Wasseranschlüssen. Hier stellt man die fundamentalen Regeln ein.

Wir konzentrieren uns hier auf die wichtigsten Punkte *neben* den Editoren.

3.1 - Database-Bereich: Functions

Man kann sich *Database Functions* wie *Methoden* vorstellen, die direkt *in* der Datenbank agieren. Da wir ja kein "richtiges" Backend haben, wird hier die Logik unserer Seite definiert.

Es ist viel effizienter, diese Logik direkt in die Datenbank zu legen.

- **Was macht man hier?** Man schreibt kleine Programme (meist in `pgSQL`, was wie eine erweiterte Version von SQL ist), die komplexe Aufgaben erledigen.
- **Beispiel:** Man könnte eine Funktion `calculate_user_score()` schreiben. Jedes Mal, wenn wir diese Funktion aufrufen, zählt sie automatisch alle Punkte in den Wochentests eines Users zusammen und gibt eine Punktzahl zurück.

3.2 - Database-Bereich: Triggers

Triggers sind das "Wenn-Dann"-System der Datenbank. Sie sind nutzlos ohne Functions.

Ein Trigger ist eine Regel, die wir auf eine Tabelle setzen. Er "lauscht" auf ein Ereignis (z.B. `INSERT`, `UPDATE`, `DELETE`) und führt dann *automatisch* eine Funktion aus.

- **Was macht man hier?** Man verknüpft ein Ereignis mit einer Funktion.
- **Beispiel:** Wir erstellen einen Trigger, der *immer wenn* (`AFTER INSERT`) ein neuer Benutzer in der `auth.users`-Tabelle erstellt wird, *automatisch* die Funktion `create_public_profile()` ausführt. Diese Funktion kopiert dann die User-ID und die E-Mail in unsere öffentliche `profiles`-Tabelle.

3.3 - Database-Bereich: Roles

Das ist die tiefste Ebene der Benutzerverwaltung. Es geht hier *nicht* um die normalen Webseiten-Nutzer (die sich per E-Mail/Passwort anmelden), sondern um die *Rollen in der Datenbank selbst*.

- **Was macht man hier?** Man legt fest, wer (welcher Dienst) überhaupt Tabellen erstellen, lesen oder die Datenbank verwalten darf.
- **Wichtig:** Als Newbies machen wir hier erstmals nichts. Man muss nur wissen: Hier werden die fundamentalen Zugriffsrechte für die Dienste selbst geregelt, die RLS (Row Level Security) nutzen diese Rollen dann.

3.4 - Database-Bereich: Extensions

Extensions sind nichts anderes als Plug-ins in Entwicklungsumgebungen.

- **Was macht man hier?** Man aktiviert Zusatzmodule, die Postgres (unserem "Betriebssystem" neue Features verleihen.
-> Deepdive in mögliche Plug-ins eventuell interessant

3.5 - Backups

WICHTIG: In der kostenlosen Version von SupaBase werden **KEINE Backups** erstellt! :((
Manuelle Backups sind kompliziert, aber möglich (Siehe Kapitel 6).

4 - Authentication

Bisher hatten wir nur die Datenbank. Den Zugriff und die Accounts regelt der Bereich Authentication.

Hier wird verwaltet, *wer* sich anmelden darf und *wie* er sich anmeldet. Dieser Dienst ist direkt mit der Datenbank verbunden und steuert die Account-Verwaltung für unsere "HSG-Lernwelt".

4.1 - Auth: Users

Das ist der einfachste Teil. Hier sehen wir eine spezielle Tabelle, die Supabase für uns verwaltet (auth.users).

- **Was macht man hier?** Man sieht jeden einzelnen Benutzer, der sich jemals registriert hat. Man sieht seine User-ID (die UUID), seine E-Mail-Adresse, wann er sich zuletzt angemeldet hat und ob er seine E-Mail bestätigt hat.

4.2 - Auth: Policies

Das hier ist das absolute Herzstück von Supabase und der Grund, warum "Auth" so mächtig ist. RLS beantwortet die Frage: "Okay, Max ist eingeloggt. Was darf er genau in der posts-Tabelle sehen?"

Was macht man hier?

Man schreibt die Regeln für *jede* Tabelle, die geschützt werden muss (also fast alle). Wenn RLS für eine Tabelle aktiviert ist (siehe Punkt 1.2), aber *keine* Policy existiert, kann **niemand** die Daten lesen.

- **Beispiel-Policy (Analogie):**
 - **Regel:** "Jeder Gast darf die Speisekarte sehen."
 - **SQL-Policy:** "Erlaube `SELECT` (Lesen) auf der `menu`-Tabelle für alle (`all users`)."
- **Beispiel-Policy (Echt):**
 - **Regel:** "Ein Benutzer darf *nur seine eigenen* Profil-Daten sehen und bearbeiten."
 - **SQL-Policy:** "Erlaube `SELECT` und `UPDATE` auf der `profiles`-Tabelle, aber nur WENN (`WHERE`) die `user_id` in der Zeile gleich der ID des aktuell eingeloggtten Benutzers ist (`auth.uid()`)."

4.3 - Auth: Providers

Hier legen wir fest, wie sich Nutzer anmelden können.

- **Was macht man hier?** Man aktiviert und konfiguriert die verschiedenen "Login-Wege".
- **Standard:** Email/Password ist meistens aktiviert. Das ist der klassische Login.
- **OAuth (Social Logins):** Hier könnten wir "Login mit Google", "Login mit Apple" etc. freischalten. (Brauchen wir für die Schule eher nicht).

4.4 - Auth: Email Templates

Wenn ein Nutzer sich registriert oder sein Passwort vergisst, muss unsere App E-Mails versenden.

- **Was macht man hier?** Man passt die Vorlagen für diese automatischen E-Mails an.
- **Beispiele:**
 - "Confirm your signup")
 - "Reset your password" (Passwort zurücksetzen)

5 - Storage

Zuletzt brauchen wir noch "Storage".

Man kann sich das wie eine Cloud-Festplatte (z.B. Google Drive oder Dropbox) vorstellen, die direkt an unsere Datenbank angeschlossen ist.

5.1 - Storage: Wozu brauchen wir das?

Unsere Datenbank (Tables) ist super für Text, Zahlen und IDs (z.B. den Namen eines Lehrmaterials). Für die Lernmaterialien und andere Dateien kommt dann Storage ins Spiel.

- **Was macht man hier?** Man speichert alle statischen Dateien, die unsere App braucht: Lehrmaterialien (PDFs), Bilder für den News-Bereich oder vielleicht Profilbilder.
- **Buckets (Ordner):** Man erstellt "Buckets", um die Dateien zu organisieren. Wir werden vermutlich einen Bucket `public_assets` für Bilder brauchen und einen geschützten Bucket `learning_materials` für die PDFs, auf die nur eingeloggte SuS/Lehrpersonen Zugriff haben.
- **Sicherheit:** Genau wie die Datenbank-Tabellen können (und müssen!) wir auch die Storage-Buckets mit Policies (Sicherheitsregeln) versehen. (z.B. "Nur Lehrpersonen dürfen in `learning_materials` hochladen.")

6 - Manuelle Backups

Wie macht man manuelle Backups?

- **Der flexible Weg (Table Editor):**
 - Geh in den `Table Editor`.
 - Wähle eine wichtige Tabelle (z.B. `profiles`).
 - Klicke oben auf `Table` und dann `Export to CSV`.
 - Das müsst ihr für JEDE Tabelle einzeln machen, aber es ist gut, um z.B. nur die User-Profile zu sichern.
- **Der Profi-Weg (Supabase CLI):**
 - Wenn ihr die Supabase CLI (Command Line Interface) nutzt, könnt ihr `supabase db dump -f backup.sql` verwenden, um die gesamte Datenbank-Struktur und die Daten zu sichern.

Regel: Wir müssen vor jeder größeren Änderung und eventuell am Ende von Team-Meetings ein manuelles Backup machen!

7 - Relationen (Foreign Keys):

Unsere ganze Logik basiert auf Verknüpfungen (Relationen).

Warum brauchen wir das?

- Wie weiß die Datenbank, welche Eltern zu wem gehören?
- Wie verknüpfen wir einen Eintrag im "Wochen-Test" mit einem bestimmten User-Profil?
- Wie sehen wir, welche Lehrkraft welches PDF im Storage hochgeladen hat?

Die Antwort ist immer: **Foreign Keys**.

Beispiel im Table Editor:

1. Wir haben unsere Tabelle `profiles`, in der alle User (SuS, Eltern, Lehrpersonen) mit ihrer einzigartigen `id` (dem Primary Key) gespeichert sind.
2. Jetzt erstellen wir eine neue Tabelle, z.B. `weekly_tests_results`
3. In dieser neuen Tabelle fügen wir eine Spalte `student_id` hinzu.
4. Wenn man diese Spalte erstellt, klickt man auf das **Ketten-Symbol** (Link), um eine "Foreign Key Relation" zu erstellen.
5. Du wählst die Tabelle `profiles` und die Spalte `id` aus.

Ergebnis: Man kann jetzt in `weekly_tests_results` nur IDs eintragen, die es auch wirklich in der `profiles`-Tabelle gibt. Supabase sorgt dafür, dass die Daten konsistent bleiben.

Das ist die Basis, um später Abfragen zu machen.

8 - Anbindung an SvelteKit

Die logische letzte Frage ist: *Wie spricht unser SvelteKit-Frontend jetzt mit dieser Datenbank?*

Das Frontend muss sich bei Supabase "ausweisen", um Daten abzufragen oder einen Login zu versuchen.

Die 3-Schritte-Anbindung:

1. API-Keys finden: Jedes Supabase-Projekt hat einzigartige "Schlüssel".

- Man geht im Supabase Dashboard zu **Project Settings**
- Dann auf **API**.
- Hier findet man die **Project URL** (z.B. `https://xyz.supabase.co`) und die **API Keys**.
- Wir brauchen den **anon** (anonymen) Key. Dieser ist "public" und darf im Frontend-Code sichtbar sein. Er erlaubt nur Aktionen, die man per RLS-Policies freigegeben hat
- **NIEMALS** den *Secret API Key* an dieser Stelle nutzen!

2. Supabase-Client installieren:

Damit SvelteKit mit Supabase sprechen kann, brauchen wir den Client mit den dazugehörigen Svelte Kit-Plugins.

Man öffnet das Terminal (z.B. in IDEA oder PHPStorm) im SvelteKit-Projektordner und tippt:
npm install @supabase/supabase-js

3. Den Client sicher initialisieren:

Man schreibt die Keys niemals direkt in den Code! Sie gehören in "Environment-Variablen".

- Man erstellt in deinem SvelteKit-Projekt eine Datei namens **.env** (falls nicht schon vorhanden).
- Man trägt dort die Keys ein (in SvelteKit müssen sie mit **PUBLIC_** anfangen, damit sie im Browser verfügbar sind):
`PUBLIC_SUPABASE_URL=https://deine-url.supabase.co`
`PUBLIC_SUPABASE_ANON_KEY=dein-langer-anon-key`
- Jetzt kann man im SvelteKit-Code (z.B. in einer zentralen `supabaseClient.js`-Datei) den Client erstellen, der auf diese sicheren Variablen zugreift.

Die genaue Initialisierung des Clients steht bereit für Copy-Paste in Supabase selbst im Abschnitt der Navigationsleiste: **API Docs**.

Nach der Erstellung des Clients, können nun Frontend-Code Befehle wie `supabase.auth.signInWithPassword()` oder `supabase.from('profiles').select('*')` genutzt werden, um mit unserer Datenbank zu interagieren.